

# Astera Solis

Documentacao tecnica da aplicacao

Laravel API, PostgreSQL, Redis, Sanctum, Next.js, Docker, GitHub Actions e VPS

|                     |             |                  |               |
|---------------------|-------------|------------------|---------------|
| Produto educacional | API Laravel | Frontend Next.js | Deploy Docker |
|---------------------|-------------|------------------|---------------|

Plataforma para gestao de escolas, colecoes didaticas, materiais digitais, quizzes e tentativas de estudantes.

# Sumario

- 01. Introducao e Contexto
- 02. Visao Geral da Solucao
- 03. Escopo Funcional e Modulos
- 04. Arquitetura Fullstack
- 05. Autenticacao com Laravel Sanctum
- 06. Modelagem de Dados
- 07. Backend Laravel
- 08. Frontend Next.js
- 09. Seguranca, Validacao e Permissoes
- 10. Deploy, CI/CD e Operacao
- 11. Testes, Qualidade e Checklist
- 12. Execucao Local e Comandos
- 13. Decisoes Tecnicas e Tradeoffs
- 14. Glossario Tecnico

# Mapa visual da aplicacao

Arquitetura em quatro blocos: experiencia web, API, dados e operacao.

|          |          |             |         |
|----------|----------|-------------|---------|
| Browser  | Next.js  | Laravel API | Sanctum |
| Policies | Eloquent | PostgreSQL  | Redis   |
| GHCR     | VPS      | nginx       | certbot |

## Fluxo de login

|                 |            |                 |             |
|-----------------|------------|-----------------|-------------|
| GET csrf-cookie | POST login | Sessao HttpOnly | GET auth/me |
|-----------------|------------|-----------------|-------------|

## Fluxo de quiz

|                  |                |                 |                 |
|------------------|----------------|-----------------|-----------------|
| Student responde | Request valida | Service calcula | Attempt gravada |
|------------------|----------------|-----------------|-----------------|

## Modelo de dados resumido

| Entidade             | Responsabilidade        | Relacionamentos principais              |
|----------------------|-------------------------|-----------------------------------------|
| <b>schools</b>       | instituicoes atendidas  | hasMany users                           |
| <b>users</b>         | usuarios e perfis       | belongsTo school, hasMany attempts      |
| <b>collections</b>   | colecoes didaticas      | hasMany materials e quizzes             |
| <b>materials</b>     | conteudos digitais      | belongsTo collection                    |
| <b>quizzes</b>       | simulados               | belongsTo collection, hasMany questions |
| <b>questions</b>     | questoes e alternativas | belongsTo quiz                          |
| <b>quiz_attempts</b> | respostas e nota        | belongsTo quiz e user                   |
| <b>seed_runs</b>     | controle de seed unico  | registro operacional                    |

# Introducao e Contexto

## Objetivo deste capitulo

Este capitulo apresenta o Astera Solis, uma aplicacao full stack para gestao de conteudos educacionais, escolas, usuarios, colecoes didaticas, materiais digitais e quizzes.

A documentacao descreve a arquitetura, os fluxos de negocio, os componentes da solucao, os modelos de dados, a seguranca, os testes e o processo de deploy.

## Contexto da aplicacao

O Astera Solis organiza a operacao digital de uma editora educacional. A plataforma centraliza conteudos e atividades avaliativas para diferentes perfis de usuario.

A escolha do tema combina com uma editora porque o dominio envolve:

- catalogo de colecoes;
- materiais digitais de apoio;
- professores e estudantes;
- simulados e avaliacao;
- controle editorial;
- gestao por escola.

## Estrutura da solucao

O projeto usa um monorepo com duas aplicacoes:

```
astera-solis-laravel-case/  
  astra-solis-api/      # API Laravel  
  astra-solis-web/     # Frontend Next.js  
  deploy/              # Compose e templates nginx de producao  
  scripts/             # Script de deploy em VPS  
  docs/                # Documentacao tecnica
```

Stack principal:

- Laravel como API;
- PostgreSQL como banco relacional;
- Redis para cache/fila;
- Laravel Sanctum com cookie HttpOnly e CSRF;
- Next.js App Router com TypeScript;
- Tailwind CSS;
- Docker e Docker Compose;
- GitHub Actions;
- GHCR;
- nginx e certbot na VPS.

## Escopo tecnico

A aplicacao foi organizada para manter responsabilidades claras entre frontend,

backend, banco de dados e infraestrutura:

- API com routes, controllers, Form Requests, policies, resources e services;
- banco modelado com migrations e relacionamentos Eloquent;
- autenticação SPA realista com Sanctum;
- permissões por perfil;
- testes de regras importantes;
- frontend consumindo a API real;
- deploy com imagens Docker e pipeline automatizado;
- seed idempotente para ambientes de homologação e apresentação.

## Leitura recomendada

Para entender a solução rapidamente:

1. Leia `02-visao-geral-da-solucao.md`.
2. Leia `04-arquitetura-fullstack.md`.
3. Leia `05-autenticacao-sanctum.md`.
4. Leia `10-deploy-ci-cd-e-operacao.md`.
5. Rode o projeto seguindo `12-execucao-local-e-comandos.md`.

# Visao Geral da Solucao

## Produto

Astera Solis e uma plataforma educacional para uma editora. Ela permite organizar conteudos didaticos, materiais digitais e atividades avaliativas por perfil de usuario.

## Perfis

```
admin:
  gerencia tudo

editor:
  cria e edita colecoes, materiais e quizzes

teacher:
  visualiza colecoes, materiais, quizzes e tentativas

student:
  visualiza materiais e responde quizzes
```

## Modulos

### Escolas

Representam instituicoes atendidas pela editora. Usuarios professores e estudantes podem estar vinculados a uma escola.

### Colecoes

Representam linhas didaticas, por exemplo uma colecao infantil, uma colecao de ciencias ou uma colecao de humanidades.

### Materiais

Representam conteudos digitais da colecao:

- ebook;
- video;
- quiz;
- PDF;
- game.

### Quizzes

Representam simulados ou avaliacoes. Cada quiz possui questoes, alternativas, gabarito, nota minima e tentativas.

### Tentativas

Quando um estudante responde um quiz, a API calcula a pontuacao, grava as respostas e informa se ele foi aprovado.

## Credenciais do ambiente inicial

Os seeders criam usuarios iniciais para acesso ao ambiente:

```
admin@astera.test / password
editor@astera.test / password
teacher@astera.test / password
student@astera.test / password
```

# Escopo Funcional e Modulos

## Objetivo deste capitulo

Este capitulo descreve os modulos funcionais do Astera Solis e como cada parte da aplicacao participa do fluxo educacional.

## Gestao de usuarios

A aplicacao trabalha com quatro perfis:

- admin;
- editor;
- teacher;
- student.

Cada usuario possui nome, email, senha, role e, quando aplicavel, vinculo com uma escola.

## Gestao de escolas

O modulo de escolas registra instituicoes atendidas pela plataforma. Cada escola possui nome, cidade, estado, codigo INEP opcional e status ativo.

Esse modulo serve como base para organizar professores e estudantes.

## Gestao de colecoes

Colecoes representam linhas didaticas da editora. Cada colecao possui:

- titulo;
- slug;
- descricao;
- segmento;
- disciplina;
- status.

Colecoes agrupam materiais e quizzes.

## Gestao de materiais

Materiais representam conteudos digitais associados a uma colecao.

Tipos previstos:

- ebook;
- video;
- quiz;
- PDF;
- game.

Cada material possui titulo, resumo, URL, tempo estimado e status.



## Gestao de quizzes

Quizzes representam avaliacoes ou simulados. Cada quiz pertence a uma colecao e possui titulo, descricao, nota minima e status.

As perguntas do quiz guardam:

- enunciado;
- alternativas em JSON;
- alternativa correta;
- pontuacao.

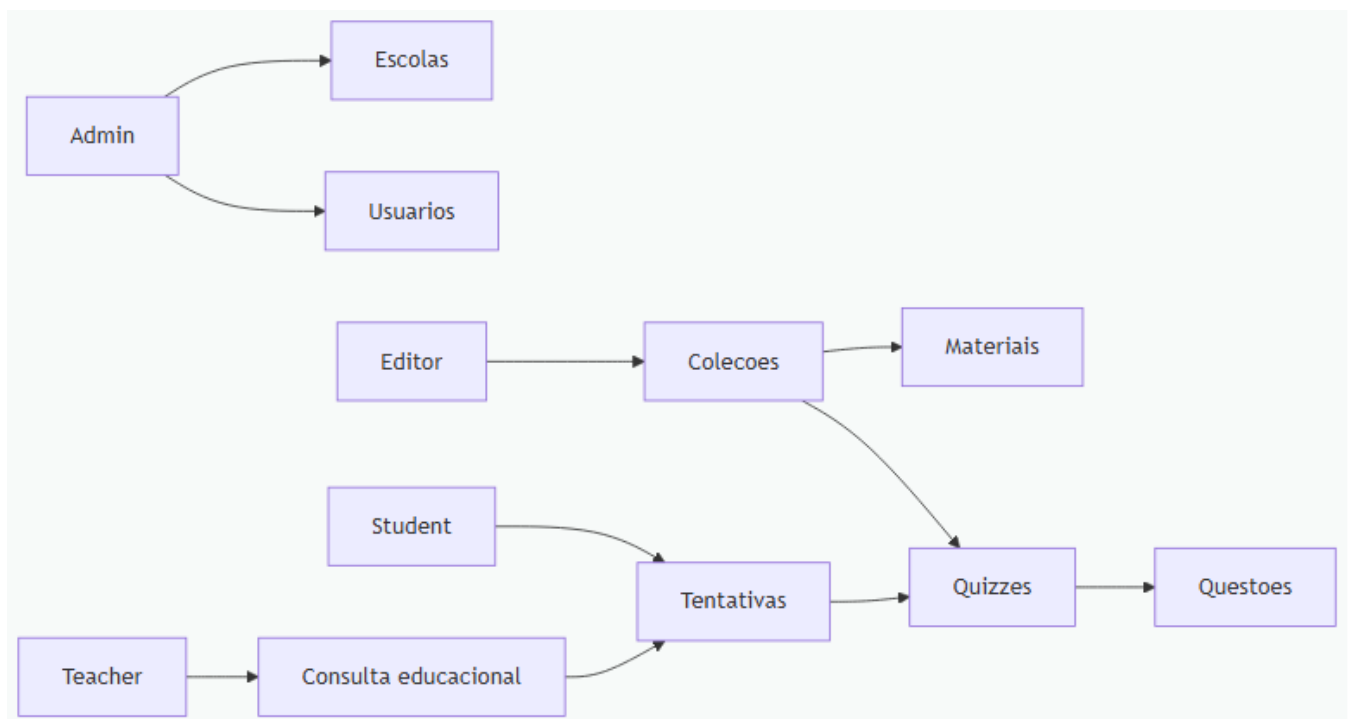
## Tentativas de quiz

Estudantes podem enviar respostas para um quiz. A API calcula a pontuacao, define aprovacao e registra a tentativa.

Cada tentativa guarda:

- usuario;
- quiz;
- respostas enviadas;
- nota calculada;
- status de aprovacao;
- data de envio.

## Fluxo funcional resumido



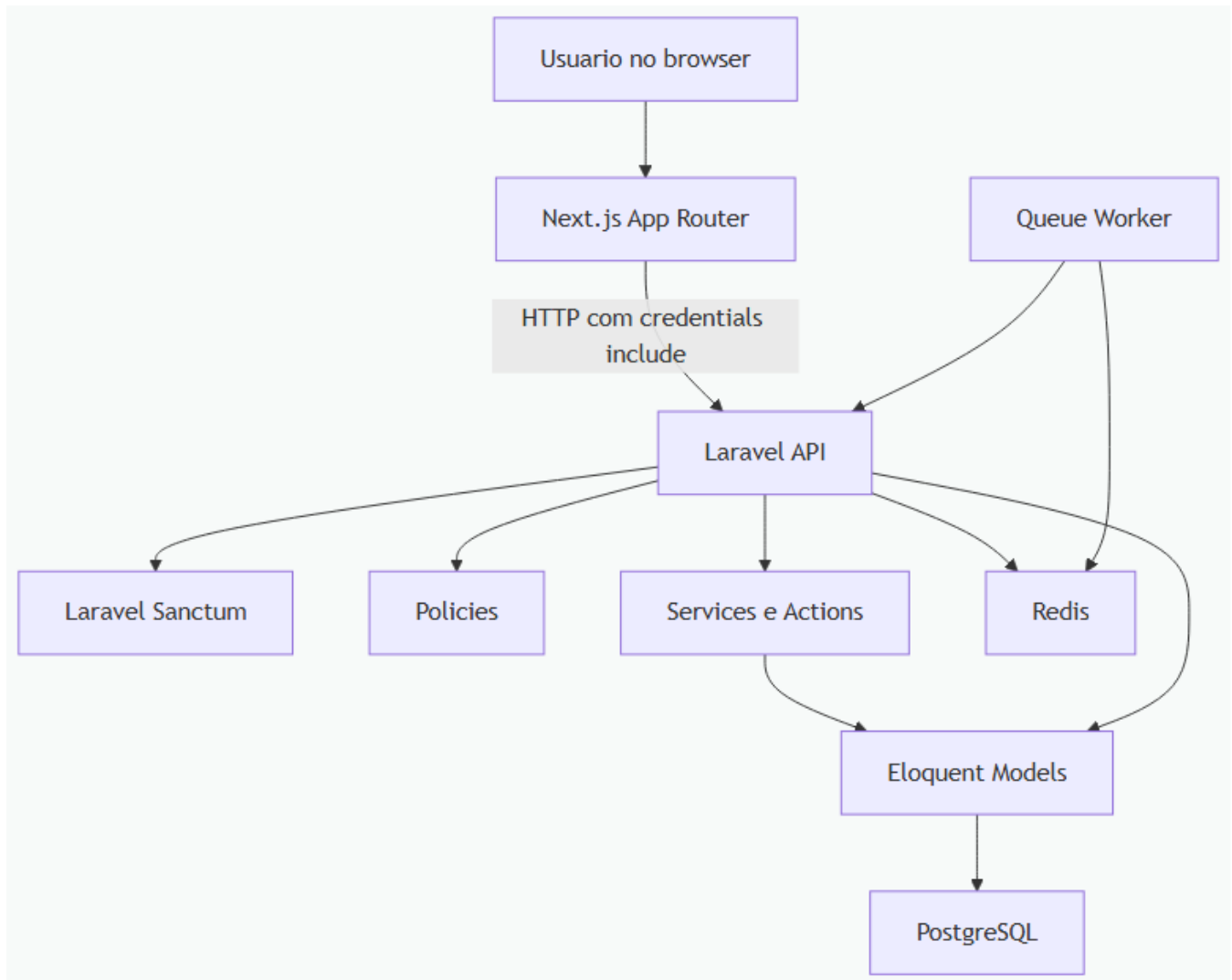
## Regras funcionais principais

- admin gerencia todos os recursos;
- editor gerencia colecoes, materiais e quizzes;

- teacher consulta conteudos e tentativas;
- student consulta materiais e responde quizzes;
- rotas de dominio exigem autenticacao;
- alteracoes de dados passam por validacao e autorizacao no backend.

# Arquitetura Fullstack

## Visao geral



## Separacao de responsabilidades

### Frontend

O frontend se concentra em:

- autenticar o usuario;
- renderizar telas;
- enviar formularios;
- mostrar estados de carregamento, erro e sucesso;
- respeitar permissoes vindas do usuario logado.

### Backend

O backend se concentra em:

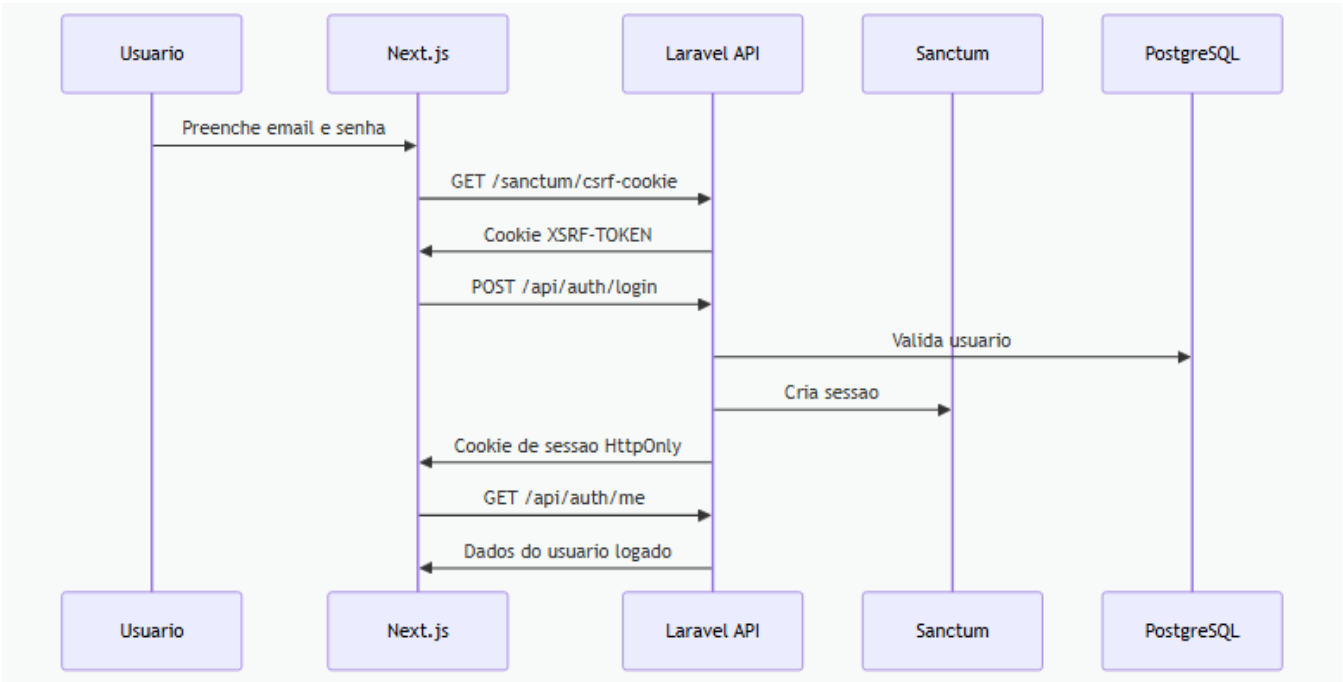
- autenticar sessoes;
- validar dados;

- autorizar acoes por perfil;
- persistir e consultar dados;
- calcular pontuacao de quizzes;
- expor JSON para o frontend.

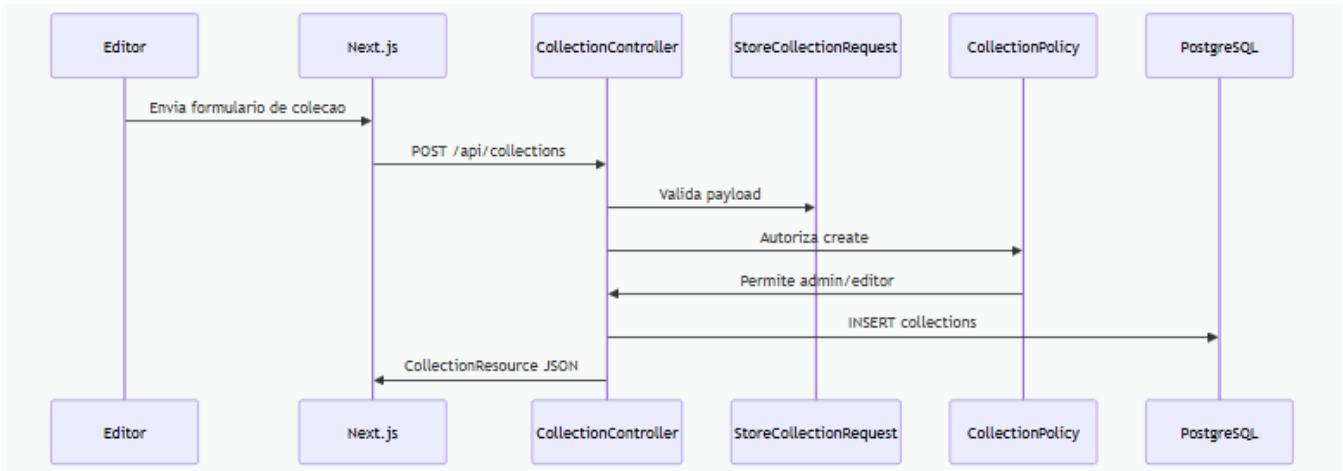
## Banco

O PostgreSQL guarda os dados relacionais. Redis fica reservado para cache e fila, principalmente no ambiente Docker/VPS.

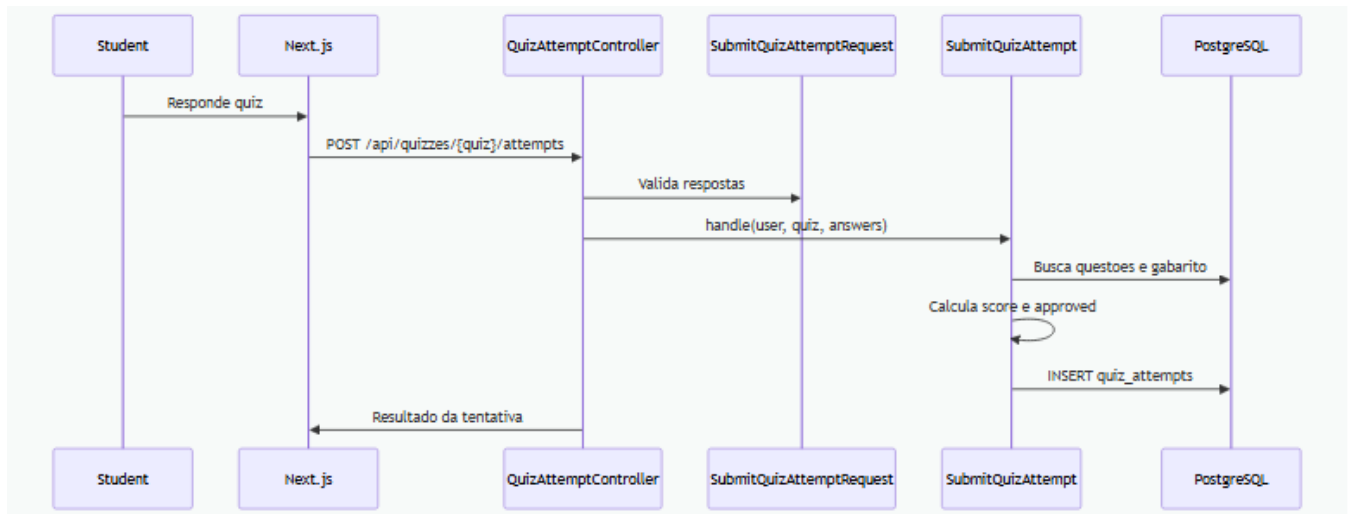
## Fluxo de autenticacao



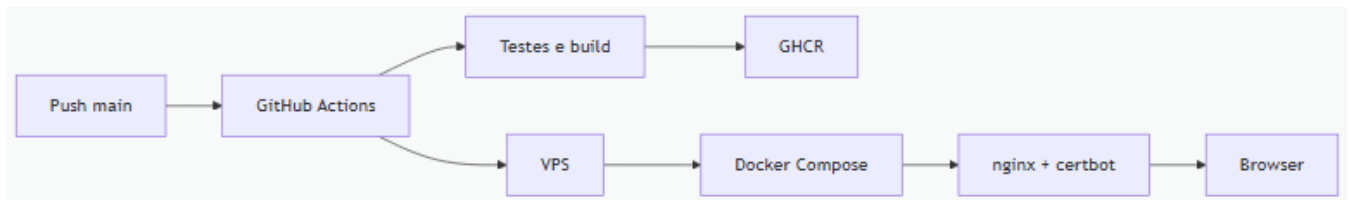
## Fluxo de criacao de colecao



## Fluxo de tentativa de quiz



## Deploy em alto nivel



# Autenticacao com Laravel Sanctum

## Escolha tecnica

O projeto usa Laravel Sanctum no modo SPA stateful, com cookie de sessao, cookie CSRF e credentials: "include" no frontend.

Essa escolha combina com o Astera Solis porque o frontend Next.js e a API Laravel pertencem ao mesmo produto, mesmo rodando em subdominios diferentes.

## Por que nao JWT com refresh token

JWT com access token e refresh token e comum em APIs publicas, apps mobile e integracoes entre sistemas independentes. Para uma SPA first-party, Sanctum com cookie costuma ser mais simples e seguro:

- o token de sessao fica em cookie HttpOnly;
- o JavaScript nao precisa guardar access token;
- Laravel protege escritas com CSRF;
- logout invalida a sessao;
- o fluxo fica mais proximo do uso comum em aplicacoes Laravel.

## Fluxo tecnico

1. Frontend chama GET /sanctum/csrf-cookie.
2. Laravel envia o cookie XSRF-TOKEN.
3. Frontend envia POST /api/auth/login com credentials: "include".
4. Laravel valida email e senha.
5. Laravel cria sessao.
6. Browser guarda cookie de sessao HttpOnly.
7. Frontend chama rotas protegidas com o cookie.

## Variaveis importantes

No ambiente de producao:

```
APP_URL=https://api.seudominio.com
FRONTEND_URL=https://app.seudominio.com
SESSION_DOMAIN=.seudominio.com
SESSION_SECURE_COOKIE=true
SESSION_SAME_SITE=lax
SANCTUM_STATEFUL_DOMAINS=app.seudominio.com
```

## Onde isso aparece no codigo

Backend:

```
routes/api.php
app/Http/Controllers/Api/AuthController.php
app/Http/Controllers/Api/MeController.php
config/sanctum.php
config/session.php
config/cors.php
```

Frontend:

astera-solis-web/src/lib/api.ts  
astera-solis-web/src/components/LoginForm.tsx

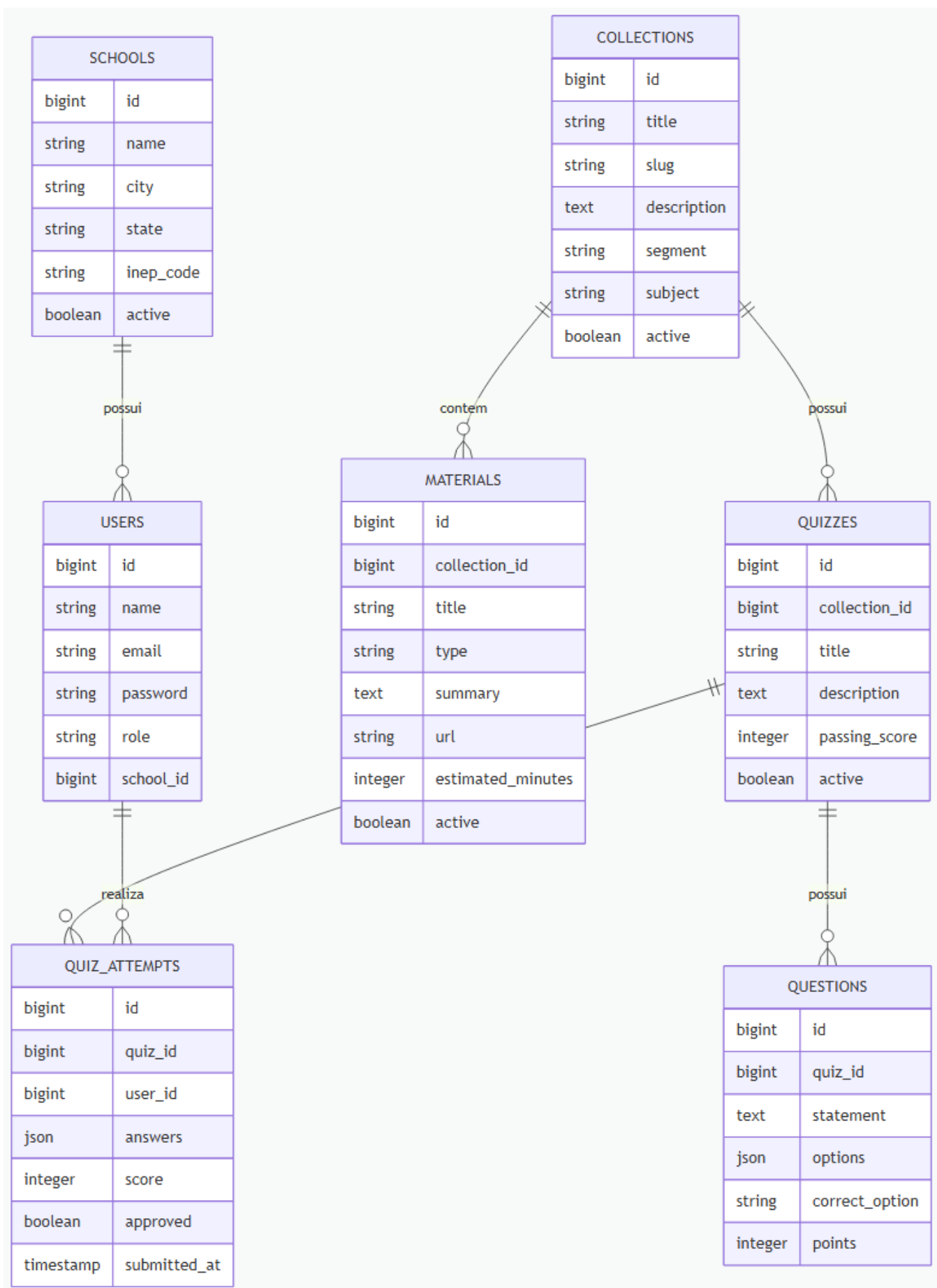
## Resumo tecnico

Sanctum foi usado no modo SPA porque o frontend e a API fazem parte do mesmo produto. O browser guarda a sessao em cookie HttpOnly, o Laravel protege requisicoes de escrita com CSRF, e o frontend usa `credentials: "include"` para enviar os cookies. Nesta arquitetura, isso evita armazenar JWT no JavaScript e mantem o fluxo bem alinhado ao ecossistema Laravel.

# Modelagem de Dados

## Entidades





## Decisoões de modelagem

## Role como string

O campo `role` usa string em vez de enum nativo do PostgreSQL. Essa escolha facilita migrations, testes e alteracoes futuras nos perfis de acesso.

## JSON em `questions.options`

As alternativas ficam em JSON para manter o quiz simples:

```
{
  "A": "Resposta A",
  "B": "Resposta B",
  "C": "Resposta C",
  "D": "Resposta D"
}
```

## JSON em `quiz_attempts.answers`

As respostas do estudante tambem ficam em JSON, mapeando `question_id` para opcao marcada.

## seed\_runs

A tabela `seed_runs` registra se o seed inicial ja foi executado no ambiente.

Isso permite popular o banco automaticamente no primeiro deploy sem duplicar dados nos deploys seguintes.

## Relacionamentos Eloquent

Principais relacionamentos:

- User belongsTo School;
- User hasMany QuizAttempt;
- School hasMany User;
- Collection hasMany Material;
- Collection hasMany Quiz;
- Material belongsTo Collection;
- Quiz belongsTo Collection;
- Quiz hasMany Question;
- Quiz hasMany QuizAttempt;
- Question belongsTo Quiz;
- QuizAttempt belongsTo Quiz;
- QuizAttempt belongsTo User.

## Casts importantes

```
// Question
'options' => 'array'

// QuizAttempt
'answers' => 'array'
'submitted_at' => 'datetime'
'approved' => 'boolean'
```

# Backend Laravel

## Estrutura principal

```
astera-solis-api/  
  app/  
    Http/  
      Controllers/Api/  
        Requests/  
        Resources/  
      Models/  
      Policies/  
      Services/  
  database/  
    migrations/  
    seeders/  
    factories/  
  routes/  
    api.php  
    console.php  
  tests/
```

## Rotas da API

Principais rotas:

```
POST    /api/auth/register  
POST    /api/auth/login  
GET     /api/auth/me  
POST    /api/auth/logout  
  
GET     /api/schools  
POST    /api/schools  
GET     /api/schools/{school}  
PUT     /api/schools/{school}  
DELETE  /api/schools/{school}  
  
GET     /api/collections  
POST    /api/collections  
GET     /api/collections/{collection}  
PUT     /api/collections/{collection}  
DELETE  /api/collections/{collection}  
  
GET     /api/materials  
POST    /api/materials  
GET     /api/materials/{material}  
PUT     /api/materials/{material}  
DELETE  /api/materials/{material}  
  
GET     /api/quizzes  
POST    /api/quizzes  
GET     /api/quizzes/{quiz}  
PUT     /api/quizzes/{quiz}  
DELETE  /api/quizzes/{quiz}  
  
POST    /api/quizzes/{quiz}/attempts
```

As rotas de dominio ficam protegidas por `auth: sanctum`.

## Controller fino

Os controllers nao concentram regra pesada. Eles fazem:

- autorizacao;
- chamada de validacao via Form Request;
- consulta ou persistencia simples;
- retorno com API Resource.

Exemplo de responsabilidade:

Controller -> autoriza -> valida -> executa -> retorna Resource

## Form Requests

Os Form Requests cuidam das regras de entrada:

- campos obrigatorios;
- tipos;
- tamanho maximo;
- existencia de relacionamentos;
- unicidade;
- dados permitidos.

Isso evita validacao espalhada nos controllers.

## API Resources

Resources controlam o JSON de saida. Isso ajuda a nao vazar estrutura interna do model e permite incluir relacoes apenas quando carregadas.

## Policies

Policies implementam regras por perfil:

- admin tem before e pode tudo;
- editor cria e edita conteudo editorial;
- teacher visualiza;
- student visualiza e responde quiz.

## Services

O service SubmitQuizAttempt concentra a regra de negocio do envio de quiz:

1. busca as questoes;
2. compara respostas com gabarito;
3. calcula pontuacao;
4. define aprovacao;
5. grava a tentativa.

Essa regra saiu do controller porque ela tem comportamento de dominio.

## Comando seed-once

Em routes/console.php, o comando app:seed-once roda o seed inicial apenas se a execucao ainda nao estiver registrada em seed\_runs.

Isso e util para a VPS:

- primeiro deploy popula o ambiente;
- proximos deploys nao duplicam registros;
- producao real poderia desligar isso por variavel.

# Frontend Next.js

## Estrutura principal

```
astera-solis-web/  
  src/  
    app/  
      dashboard/  
      collections/  
      materials/  
      quizzes/  
      login/  
    components/  
    lib/
```

## App Router

O frontend usa o App Router do Next.js. As telas ficam dentro de `src/app` e compartilham componentes de layout, cards, badges, formulario de login e componentes de feedback.

## Consumo da API

O arquivo `src/lib/api.ts` define o cliente HTTP base:

- usa `NEXT_PUBLIC_API_URL`;
- envia `credentials: "include"`;
- busca CSRF antes de escritas;
- inclui `X-XSRF-TOKEN` quando necessario.

## Leituras

As leituras usam helpers em `src/lib/read-api.ts`, que normalizam retorno da API Laravel, paginação e erros.

## Escritas

As escritas usam `src/lib/mutations.ts`, chamando:

```
POST  
PUT  
PATCH  
DELETE
```

Antes de cada escrita, o frontend chama o fluxo CSRF.

## Telas

### Login

Permite entrar com os usuarios de seed e inicia o fluxo Sanctum.

### Dashboard

Carrega usuario logado, colecoes, materiais e quizzes diretamente da API.

## Colecoes

Lista colecoes e permite criar, editar e excluir quando o perfil tem permissao.

## Materiais

Lista materiais associados as colecoes e permite operacoes conforme permissao.

## Quizzes

Lista quizzes e permite criar, editar, excluir e responder.

## Detalhe do quiz

Mostra questoes, alternativas e envia tentativa para a API calcular a nota.

## Interface

A interface usa Tailwind com componentes proprios. O foco visual e ser claro para uso operacional: cards, tabelas, badges de status, formularios e feedback de erro/sucesso.

## Integracao com a API

- O frontend nao usa dados mockados nas telas principais.
- A API Laravel e a fonte real dos dados.
- O login depende do cookie Sanctum.
- A permissao visual acompanha a role do usuario logado.
- As operacoes de escrita usam CSRF.

# Seguranca, Validacao e Permissoes

## Camadas de protecao

O projeto protege a aplicacao em varias camadas:

- autenticao com Sanctum;
- CSRF nas requisicoes de escrita;
- cookies HttpOnly para sessao;
- CORS configurado para frontend;
- Form Requests para validacao;
- Policies para autorizacao;
- senha com hash automatico no model User;
- dados sensiveis escondidos via Hidden.

## Autenticacao

Rotas publicas:

```
POST /api/auth/register
POST /api/auth/login
```

Rotas protegidas:

```
GET /api/auth/me
POST /api/auth/logout
apiResource schools
apiResource collections
apiResource materials
apiResource quizzes
POST /api/quizzes/{quiz}/attempts
```

## Matriz de permissoes

```
admin:
  tudo

editor:
  visualiza e gerencia colecoes, materiais e quizzes

teacher:
  visualiza dados educacionais

student:
  visualiza conteudo e envia tentativa de quiz
```

## Por que usar policies

Policies deixam a autorizacao perto do recurso. Em vez de espalhar `if role` por controllers e componentes, cada model tem uma classe de regra.

Exemplo conceitual:

```
public function create(User $user): bool
{
    return $user->isEditor();
}
```

Com before, o admin consegue passar por todas as regras:

```
public function before(User $user, string $ability): ?bool
{
    return true;
}
```

```
    return $user->isAdmin() ? true : null;  
}
```

## Validacao

Form Requests validam dados antes da persistencia. Isso facilita respostas 422 padronizadas para o frontend.

Exemplos:

- StoreCollectionRequest;
- UpdateCollectionRequest;
- StoreMaterialRequest;
- UpdateMaterialRequest;
- StoreQuizRequest;
- UpdateQuizRequest;
- SubmitQuizAttemptRequest;
- LoginRequest;
- RegisterRequest.

## Erros esperados

401 Unauthorized:  
usuario nao autenticado

403 Forbidden:  
usuario autenticado, mas sem permissao

422 Unprocessable Entity:  
payload invalido

204 No Content:  
exclusao realizada com sucesso

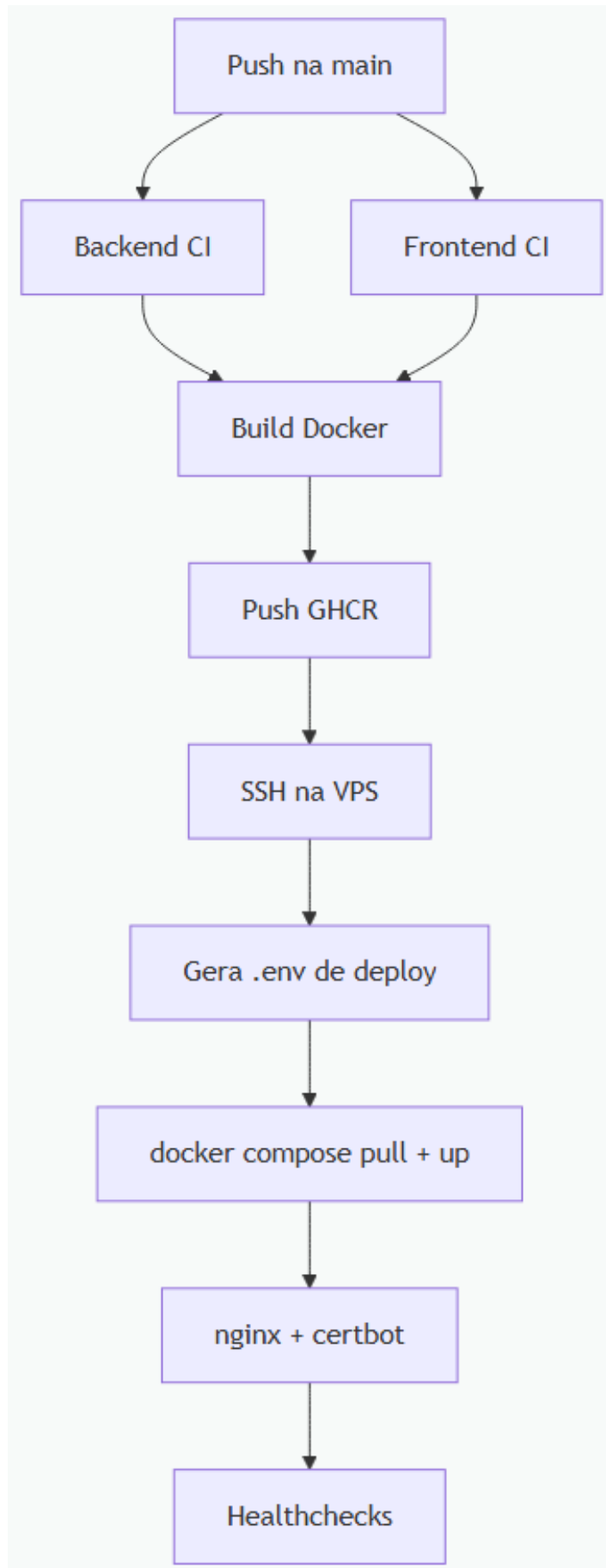


# Deploy, CI/CD e Operacao

## Objetivo

Este capitulo descreve o deploy em VPS com Docker, GitHub Actions, GHCR, nginx e certbot.

## Pipeline



## CI backend

O job Laravel executa:

```
composer install --no-interaction --prefer-dist
cp .env.example .env
php artisan key:generate --ansi
php artisan test
vendor/bin/pint --test
```

## CI frontend

O job Next.js executa:

```
npm ci
npm run lint
npm run build
```

## Imagens

As imagens publicadas no GHCR seguem o formato:

```
ghcr.io/cledson96/astera-solis-laravel-case-api:<sha>
ghcr.io/cledson96/astera-solis-laravel-case-web:<sha>
```

Tambem existe tag latest.

## Compose de producao

Servicos:

- api;
- web;
- db;
- redis;
- worker.

O banco e o Redis nao ficam expostos publicamente. API e web sao publicados apenas em 127.0.0.1, e o nginx do host faz o proxy para os dominios.

## Seed inicial em deploy

O deploy define:

```
RUN_MIGRATIONS_ON_DEPLOY=true
RUN_SEED_ON_DEPLOY=true
```

No primeiro deploy, a API roda migrations e app:seed-once. O comando grava uma linha em seed\_runs. Nos proximos deploys, ele detecta que ja executou e nao duplica os dados iniciais.

## Secrets obrigatorios

```
VPS_HOST
VPS_USER
VPS_SSH_KEY
APP_KEY
POSTGRES_PASSWORD
```

## Vars obrigatorias

```
FRONTEND_DOMAIN
API_DOMAIN
SESSION_DOMAIN
```

## Vars com defaults

```
DEPLOY_PATH=/opt/astera-solis
COMPOSE_PROJECT_NAME=astera-solis-production
FRONTEND_PORT=3000
API_PORT=8000
POSTGRES_DB=astera_solis
POSTGRES_USER=astera
```

## Operacao na VPS

Comandos uteis:

```
cd /opt/astera-solis
docker compose -f deploy/docker-compose.vps.yml ps
docker compose -f deploy/docker-compose.vps.yml logs -f api
docker compose -f deploy/docker-compose.vps.yml logs -f web
docker compose -f deploy/docker-compose.vps.yml logs -f worker
sudo nginx -t
sudo systemctl reload nginx
sudo certbot certificates
```

# Testes, Qualidade e Checklist

## Testes backend

O backend possui testes feature para validar os fluxos mais importantes:

- login com CSRF;
- login invalido;
- rota protegida sem sessao;
- logout;
- usuario autenticado consultando perfil;
- estudante impedido de criar colecao;
- editor criando material;
- admin gerenciando escolas;
- estudante enviando tentativa de quiz;
- seed-once executando apenas uma vez.

## Comandos

```
cd astera-solis-api
php artisan test
vendor/bin/pint --test
php artisan route:list
```

Frontend:

```
cd astera-solis-web
npm run lint
npm run build
```

Docker:

```
docker compose config
docker compose -f deploy/docker-compose.vps.yml config
```

## Checklist antes de apresentar

```
[ ] API sobe sem erro
[ ] Frontend sobe sem erro
[ ] Login admin funciona
[ ] Login editor funciona
[ ] Login student funciona
[ ] Dashboard carrega dados reais
[ ] Colecoes listam dados da API
[ ] Materiais listam dados da API
[ ] Quizzes listam dados da API
[ ] Student consegue responder quiz
[ ] Editor consegue criar conteudo
[ ] Student nao consegue criar colecao
[ ] Testes Laravel passam
[ ] Build Next.js passa
[ ] Deploy Actions esta verde
[ ] HTTPS esta ativo nos dominios
```

## Roteiro de validacao funcional

1. Entrar como admin@astera.test.
2. Mostrar dashboard com dados vindos da API.
3. Abrir colecoes e explicar relacionamento com materiais e quizzes.
4. Entrar como editor@astera.test.

5. Criar ou editar uma colecao/material.
6. Entrar como student@astera.test.
7. Responder um quiz.
8. Mostrar pontuacao calculada pelo backend.
9. Explicar rapidamente o fluxo Sanctum.
10. Mostrar GitHub Actions e deploy em VPS.

## Perguntas que podem aparecer

## Observacoes de qualidade

### Uso do Laravel

Laravel concentra API, validacao, ORM, migrations, policies, testes e autenticacao, mantendo o backend organizado em camadas conhecidas do ecossistema.

### MVC na API

Models representam dados e relacionamentos, controllers recebem as requisicoes, e a camada de apresentacao no backend e feita por API Resources. Como o frontend e Next.js, as views Blade nao fazem parte do fluxo principal.

### Regra de negocio

Parte esta nos models e policies, e a regra de calculo de tentativa de quiz esta no service SubmitQuizAttempt.

### Banco relacional

O dominio e relacional: usuarios pertencem a escolas, colecoes possuem materiais e quizzes, quizzes possuem questoes e tentativas.

### Containerizacao

Docker padroniza ambiente local e deploy. A VPS nao precisa buildar nada, ela baixa imagens prontas do GHCR.

# Execucao Local e Comandos

## Subir banco e Redis

Na raiz do projeto:

```
cd C:\trabalho\astera-solis-laravel-case
docker compose up -d
```

## Backend Laravel

```
cd C:\trabalho\astera-solis-laravel-case\astera-solis-api
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate
php artisan db:seed --class=DemoSeeder
php artisan serve
```

API local:

<http://localhost:8000>

## Frontend Next.js

```
cd C:\trabalho\astera-solis-laravel-case\astera-solis-web
npm install
npm run dev
```

Frontend local:

<http://localhost:3000>

## Variavel local do frontend

Criar `.env.local` no frontend:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

## Variaveis locais importantes da API

Exemplo para PostgreSQL local via Docker:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=astera_solis
DB_USERNAME=astera
DB_PASSWORD=astera

SESSION_DRIVER=database
SESSION_DOMAIN=null
SANCTUM_STATEFUL_DOMAINS=localhost:3000,127.0.0.1:3000
FRONTEND_URL=http://localhost:3000
```

## Recriar banco local

Apaga as tabelas e roda tudo de novo:

```
php artisan migrate:fresh --seed
```

Para rodar o seed inicial especifico:

```
php artisan db:seed --class=DemoSeeder
```

## Validacoes

```
php artisan test  
vendor/bin\pint --test  
php artisan route:list
```

```
npm run lint  
npm run build
```

## Logins iniciais

```
admin@astera.test / password  
editor@astera.test / password  
teacher@astera.test / password  
student@astera.test / password
```



# Decisoes Tecnicas e Tradeoffs

## Monorepo

O projeto usa monorepo para manter API, frontend, deploy e documentacao juntos.

Vantagens:

- facilita avaliacao;
- simplifica o deploy da aplicacao;
- centraliza documentacao;
- permite pipeline unico.

Tradeoff:

- jobs do CI precisam separar backend e frontend por pasta.

## Laravel como API

Laravel foi usado como API JSON. Blade nao e a camada principal de interface.

Vantagens:

- ecossistema robusto;
- Sanctum integrado;
- Eloquent e migrations;
- Form Requests;
- Policies;
- testes feature produtivos.

Tradeoff:

- exige configurar CORS, Sanctum e cookies corretamente para SPA.

## Sanctum cookie

Sanctum cookie foi escolhido em vez de JWT.

Vantagens:

- bom para SPA first-party;
- cookie HttpOnly reduz exposicao no JavaScript;
- CSRF ja se integra ao Laravel;
- fluxo simples para login/logout.

Tradeoff:

- exige cuidado com dominio, subdominio, SameSite e CORS.

## Next.js

Next.js foi escolhido para o frontend por combinar TypeScript, roteamento, componentizacao e build de producao.

Vantagens:

- stack valorizada no mercado;
- App Router;

- bom suporte a TypeScript;
- deploy standalone com Docker.

Tradeoff:

- para Sanctum, e preciso tratar cookies e credentials: "include" com atencao.

## PostgreSQL

PostgreSQL foi escolhido por ser um banco relacional forte e comum em producao.

Vantagens:

- relacionamentos claros;
- integridade referencial;
- JSON quando necessario;
- bom encaixe com Laravel.

Tradeoff:

- ambiente local precisa de Docker ou instalacao propria.

## Redis

Redis entra como cache/fila no compose.

Vantagens:

- habilita infraestrutura de cache e filas;
- permite queue worker;
- prepara o projeto para jobs.

Tradeoff:

- adiciona mais um servico para operar.

## Seed automatico no primeiro deploy

O seed roda em deploy com controle por seed\_runs.

Vantagens:

- ambiente inicial nasce com dados;
- login admin/editor/student funciona apos primeiro deploy;
- deploys seguintes nao duplicam registros.

Tradeoff:

- em producao real, seeds automaticos precisam ser avaliados com cuidado.

## Docker na VPS

As imagens sao buildadas no GitHub Actions e publicadas no GHCR. A VPS apenas faz pull e sobe containers.

Vantagens:

- deploy mais previsivel;
- VPS nao precisa Node, Composer ou build local;

- rollback fica mais simples por tag de imagem.

Tradeoff:

- exige configurar GHCR, secrets, vars, Docker e nginx.

# Glossario Tecnico

## Laravel

Framework PHP para criar aplicacoes web, APIs, jobs, comandos, testes, migrations e integracoes.

## MVC

Padrao que separa Model, View e Controller. No Astera Solis, o Laravel usa Models e Controllers, mas a View principal e o frontend Next.js. No backend, a resposta visual e substituida por API Resources em JSON.

## Eloquent

ORM do Laravel. Permite trabalhar com tabelas como models PHP e criar relacionamentos como hasMany, belongsTo e belongsToMany.

## Migration

Arquivo versionado que cria ou altera estrutura do banco. E o historico de schema do projeto.

## Seeder

Classe que popula o banco com dados iniciais.

## Form Request

Classe do Laravel usada para validar entrada de dados antes do controller executar a regra.

## Policy

Classe que centraliza autorizacao de acoes sobre um model.

## API Resource

Classe que transforma models em JSON de resposta.

## Sanctum

Pacote Laravel para autenticar SPAs, mobile apps e tokens simples. Neste projeto, foi usado no modo SPA com cookie.

## CSRF

Protecao contra requisicoes falsas vindas de outros sites. Em escritas, o frontend envia o token XSRF para o Laravel validar.

## **Cookie HttpOnly**

Cookie que o JavaScript não consegue ler diretamente. Isso ajuda a reduzir risco em ataques XSS.

## **Redis**

Banco em memória usado para cache, filas e dados temporários.

## **GHCR**

GitHub Container Registry. Guarda as imagens Docker geradas pelo GitHub Actions.

## **FrankenPHP**

Runtime moderno para rodar aplicações PHP em container, usado na imagem de produção da API.

## **Next.js App Router**

Modelo de roteamento do Next.js usado para organizar páginas, layouts e componentes da aplicação web.

## **Tailwind CSS**

Framework CSS utilitário usado para criar a interface do frontend com classes componíveis.

## **Docker Compose**

Ferramenta usada para orquestrar API, frontend, PostgreSQL, Redis e worker em ambientes locais ou de VPS.

## **GitHub Actions**

Plataforma de automação usada para executar testes, lint, build das imagens Docker e deploy na VPS.